

What are Functions?

JAVASCRIPT

```
console.log("I am Ahmed");  
console.log("I am 20 years old");  
console.log("I am Backend developer");
```

```
console.log("I am Ahmed");  
console.log("I am 20 years old");  
console.log("I am Backend developer");
```

```
console.log("I am Ahmed");  
console.log("I am 20 years old");  
console.log("I am Backend developer");
```

```
console.log("I am Ahmed");  
console.log("I am 20 years old");  
console.log("I am Backend developer");
```

- احنا عندنا مشكلة في الكود ده انه بيتكرر كثير
- و لو حبيننا نعدل اي حاجة فيه هنضطر نعدل في كذا مكان
- عشان كدة بنستعمل الفانكشن
- بنكتب الكود مرة واحدة بس و كل ما نحتاجه بنستدعي الفانكشن فالكود بيتنفذ
- ده قلل التكرار + قلل حجم الكود يعني مبقاش سطور كثير متكررة
- ممكن نستخدم الفانكشن اكثر من مرة و لو حبيننا نعدل في الكود هنعدل مرة واحدة بس مش اكثر من مرة
- اي تعديل في الفانكشن هيسمع في كل حنة استدعيناها فيها بدون الحاجة لاننا نغير الكود في اكثر من مكان
- بالفانكشن الكود هيتحول معانا لكدة

JAVASCRIPT

```
function introduceYourself() {  
    console.log("I am Ahmed");  
    console.log("I am 20 years old");  
    console.log("I am Backend developer");  
}  
  
introduceYourself();
```

```
introduceYourself();
introduceYourself();
introduceYourself();
```

- الكود بقى اسهل في القراءة
- عدد سطور اقل

Function Syntax

```
function name() {
  // code to be executed
}
```

JAVASCRIPT

- اسم الفانكشن ليه قواعد للتسمية و هي نفس قواعد تسمية ال variables
- اسم الفانكشن لازم يكون بيعبر عنها
-
- طب كدة الفانكشن عبارة عن كود واحد بيتنفذ في كل مرة بنستدعي الفانكشن
- طب مفيش اي امل نخلي الكود ده داناميك شوية
- يعني مثلا مثلا هنعمل فانكشن بتجيب ضرب $5 * 4$

```
function Multiply4x5() {
  console.log("4 x 5 = " + (4*5));
}
```

JAVASCRIPT

- كدة الفانكشن ده مبتعملش حاجة غير انها بتضرب الرقمين دول في بعض بس
- طب عايزين نوسع استخدامها اكثر ازاى نقدر نعمل كدة
- في حاجة اسمها parameters ممكن نديها للفانكشن
- ال parameters هي عبارة عن variables بنستخدمها جوا الفانكشن بس

```
function MultiplyTwoNumbers(num1,num2) {
  console.log(`${num1} x ${num2} = ${num1*num2}`);
}
```

JAVASCRIPT

- كدة بقينا نقدر نستخدم الفانكشن اكثر من مرة بحيث نضرب اي رقمين في بعض
- محتاجين نفرق بين حاجتين ايه هي ال parameters و ايه هي ال arguments
- ال parameters هي عبارة عن ال variables اللي بنديها للفانكشن و احنا بنعملها
- ال arguments هي عبارة عن ال values اللي بنديها للفانكشن و احنا بنستدعيها

- طب ايه اللي هيحصل لو ادينا الفانكشن argument واحد بس ؟
- هتجعلنا NaN طب ليه ؟
- عشان ال parameter الثاني مفيهوش حاجة فيكون undefined
- فلما نضرب اي رقم في undefined بيرجعلنا NaN
- في النهاية وصلنا لان اي parameter مديناوش قيمة بيكون قيمته undefined
- طب ما undefined و NaN دول يعتبرو مشكلة و مينفعش يظهر و طب ايه الحل ؟
- طب ما نديه قيمة ؟ مش قولنا ال parameter عبارة عن variable
- يبقى ينفع نديه قيمة افتراضية
- لما نعمل كدة لو ال parameter ده مديناوش قيمة و احنا بنستدعي الفانكشن هياخد القيمة ده تلقائي

JAVASCRIPT

```
function MultiplyTwoNumbers(num1 = 0, num2 = 0) {
  console.log(`${num1} x ${num2} = ${num1*num2}`);
}
```

- طب دلوقتي في مشكلة بسيطة افرض محتاجين القيمة اللي هتطلع ده في كذا حنة في الصفحة
- و كل مكان عايزين نطبعها بطريقة مختلفة
- مرة زي ما هي كدة و مرة مثلا "The result is (ans)" و مرة مثلا عايزين نطبع الرقم لوحده هنعمل ايه ؟
- في حاجة اسمها return
- هنخليها بس ترجع قيمة واحدة اللي هي الناتج و احنا ناخذ القيمة و نطبعها براحتنا في اي حنة

JAVASCRIPT

```
function MultiplyTwoNumbers(num1 = 0, num2 = 0) {
  let result = num1 * num2;
  return result;
}

console.log("8 x 4 = " + MultiplyTwoNumbers(8,4));
console.log("The result is " + MultiplyTwoNumbers(8,4));
```

- طبعا نقدر بدل ما نستخدم القيمة في الطباعة علطول نحطها في variable
- بحيث لو عايزين نعمل عليها اي عمليات تانية

JAVASCRIPT

```
function MultiplyTwoNumbers(num1 = 0, num2 = 0) {
  let result = num1 * num2;
  return result;
}

let x = MultiplyTwoNumbers(8,4);
console.log("8 x 4 = " + x);
console.log("The result is " + x);
```

- عندنا فانكشنز جاهزة كثير بنستخدمها من اول ما ابتداينا بترجلا قيمة معينة زي مثلا `reverse` بتاخذ ارراي و بترجلنا ارراي جديدة فيها نفس الارراي القديمة بس معكوسة

•

- في فانكشن خدناها بس مبنترجش اي قيمة ؟

• `console.log()`

•

•

•

- طب لو عندنا عدد منعروش من ال `parameters`
- زي مثلا اللي هيسندعي الفانكشن ده عايز يجمع شوية ارقام
- انا معرفش عدد الارقام ده ايه ؟ طب ازاي اتعامل مع عدد مجهول بالنسبالي ؟
- ممكن حد يقول مثلا اننا ناخداه كارراي و نتعامل معاها كارراي و ده صح
- بس هناخداه ك `parameters` عادية و نتعامل معاها كارراي
- ازاي ؟

JAVASCRIPT

```
function sum(...args) {
  let sum = 0;
  for (let arg of args) sum += arg;
  return sum;
}

let x = sum(4, 9, 16, 25, 29, 100, 66, 77);
```

- ممكن في طريقة تانية و هي اننا نستعمل `built-in object` اسمه `arguments`
- هو برودو بيكون عبارة عن ارراي فيها كل ال `arguments` اللي بتتبع و احنا بنسندعي الفانكشن
- طب نستخدم ايه اكيد الاتنين مش شبه بعض بالظبط و الا مكانوش هيقو موجودين الاتنين
- في فرق بسيط جدا و هو ان اول طريقة ممكن نعمل `parameters` عادية و بعدين نحط ال `rest parameter` بحيث ناخذ باراميتز بالطريقة العادية و لو عايزين الباقي يكونو في ارراي
- انما ال `built-in object` ده بيبقى جواه كل الباراميترز كارراي كدة

س

Anonymous Function

طيب هندخل على حاجة جديدة

- لو عملنا كدة هيجصل ايه ؟

```
function test() {
  console.log("test");
}

let x = test();
console.log(x);
```

- ايه الناتج من هنا ؟

-

- الاجابة الفانكشن هتتفد عادي جدا

- طب ال variable هيكون محتواه ايه ؟

- هيكون محتواه عبارة عن undefined

-

- طب ليه ؟ عشان اللي اتخزن في الفانكشن هو اللي رجع منها

- يعني الفانكشن كلها اتنفذت و رجعت نتيجة لتنفيذها ده و النتيجة ده هي اللي اتخزنت في ال x

- و الفانكشن ده اصلا مفيهش return و المفروض عرفنا ان اللي مفيهش return بترجع ايه ؟

-

-

- طب لو عملنا كدة هيحصل ايه ؟

```
function test() {
  console.log("test");
}

let x = test;
x();
```

- الفانكشن زيبا زي اي value ممكن نخطها في variable

- لو جينا طبعنا ال x هنلاقي جواه الفانكشن نفسها اللي احنا عاملينها فوق

- و دلوقتي بقينا نقدر ن call الفانكشن بطريقتين باسمها الحقيقي او باسم ال variable

-

- طب ما دام كدة ليه محتاجين نعمل فانكشن و نخطها في variable و كدة ممكن نخطها علطول

```
let x = function () {
  console.log("test");
}
```

- هل كدة هتفرق ؟ ابدأ بس وفرنا على نفسنا كتابة و الفانكشن بقى ليها اسم واحد

- طب ليه اصلا بنخزن الفانكشن ؟ ماحنا نقدر نكتبها عادي و نستدعيها عادي

- الاجابة عشان نقدر نستفيد من بعض خواص الفانكشن زي ما بنستفيد من خواص ال string

JAVASCRIPT

```
console.log(x.name); // name of the function
console.log(x.length); // how many parameters in the function
```

- و في فائدة ثانية للموضوع
- احنا قولنا ان الفانكشن زي اي variable عادي مش ممكن طب نبعثها ك parameter لفانكشن ثانية

JAVASCRIPT

```
function call(functionToCall) {
  console.log("from call :");
  functionToCall();
}

call(x);
call(function() {
  console.log("function To Call");
})
```

- دلوقتي الفانكشن اللي اسمها call بقت higher order function
- الاسم ده بيطلق على الفانكشنز اللي بتاخذ في الباراميترز بتاعتها فانكشن ثانية
- و الفانكشن اللي بعناها في الباراميترز ده اسمها callback function
- اي فانكشن بنبعثها في باراميتر لفانكشن ثانية بيبقى اسمها callback function
- ال callback function هي عبارة عن فانكشن مش انا اللي بستدعيها بنفسي لا فانكشن ثانية هي اللي بتقرر تستدعيها امتى
- نجرب مثال عال callback
- عايزين نعمل فانكشن مثلا بتجمع ال total price بتاع order معين و لو ال total اكبر من 200 هتعمل خصم 10%

JAVASCRIPT

```
function calculateTotalPrice(callback, ... prices) {
  let total = 0;
  for(let price of prices) {
    total += price;
  }
  console.log(total)
  if(total > 200) {
    return callback(total);
  }
  return total;
}

function discount(total) {
  return (total - total * 0.1);
}
```

```
console.log(calculateTotalPrice(discount,10,20,50,70,270));
```

- طب نجرب كام فانكشن جاهزة كدة نثبت بيهم فكرة ال callback شوية
- اللي جاية ده مش لازم تركزو هي بتعمل ايه كدة كدة هيجي وقتها

```
setTimeout(function() {  
    console.log("anything");  
},5000);
```

JAVASCRIPT

- هي بس بتعمل فانكشن معينة بعد مرور وقت معين من تحميل البيج اللي احنا فيها
- هتستنى 5 ثواني و بعدين تنفذ الفانكشن اللي اديناها لها
- لاحظو اننا عرفنا الفانكشن بس مستدعيناهاش
- مين اللي استدعاها ؟ فانكشن `setTimeout` نفسها هي اللي استدعتها بعد مرور الوقت اللي حددناه
-
-
- الفانكشن الثانية ده بقى نركز فيها لانها مهمة هي متعلقة بالاراي

```
let arr = [1,2,1,2];  
arr.forEach(function(element){  
    console.log(element);  
});
```

JAVASCRIPT

٥

Arrow Function

- في طريقة تانية لتعريف الفانكشن معمولة عشان تختصر الكتابة شوية
- كنا بنعرف الفانكشن ازاى

```
const x = function (y,z) {  
    console.log("function x");  
}
```

JAVASCRIPT

- دلوقتي نقدر نعرفها كدة

JAVASCRIPT

```
const x = (y,z) => {
  console.log("function x");
}
```

- عنوان السهم ده بقى اسمها arrow functions

•

- نقدر برده نختصر فيها اكثر
- لو الفانكشن كل وظيفتها انها تعمل `return` لحاجة ممكن نكتبها كدة

JAVASCRIPT

```
const x = (y,z) => y * z;
```

- في طريقة ابسط خالص بس لما يكون عندنا باراميتز واحد بس

JAVASCRIPT

```
const x = y => y*2;

const x = y => {
  console.log(y);
}
```



Function Scope

- ال variable هيتشاف ازاى لو هو اتعرف جوا الفانكشن ؟

JAVASCRIPT

```
function test() {
  let x = 10;
  console.log(x);
}
test(); // 10
console.log(x); // Error
```

- المتغير اللي بيتعرف جوا الفانكشن مبيطلعش براها

•

- اي متغير بنعرفه برا نقدر عادي نستخدمه جوا الفانكشن و العكس غلط

```
let x = 10;

function test() {
  x = 20;
  console.log(x);
}

test(); // 20
console.log(x); // 20
```

- اشتغل عادي لان ال `x` متعرف برا

-
-

- طب ايه الناتج بتاع الكود ده

```
let x = 10;

function test() {
  let x = 20;
  console.log(x);
}

test(); // 20
console.log(x); // 10
```

- كل `x` فيهم في `scope` لوحدها فهم يعتبرو اصلا مش نفس ال `variable`
- جوا الفانكشن هي خدت ال `x` الاقرب ليها و اشتغلت عليها اللي هي ال `x` اللي اتعرفت في الفانكشن
- برا الفانكشن هي مش شايعة غير ال `x` اللي قيمتها 10 بس مش شايعة ال `x` الثانية لانها جوا فانكشن



Higher Order Functions

- كنا اتكلمنا من شوية عن ال `higher order functions` و قولنا ان هي الفانكشنز اللي بتاخذ فانكشن تانية كباراميتير ليها

-

- و خدنا مثال اللي هو الفانكشن اللي متعلقة بالارراي اللي هي `forEach`

```
let arr = [1,2,1,2];
arr.forEach((element,index) => {
  console.log(element);
});
```

- الفانكشن ده بتلوب عالاراي و بتطبق الفانكشن اللي انت هتديهولها على كل عنصر جوا
- الفانكشن اللي هي ال `callback` فانكشن بتاخذ باراميترز معينة ممكن نديهالها او لا بنسميهم اي اسماء عادي
- اول باراميتر بيتحط فيه كل عنصر في الاراي عشان يتطبق عليه الفانكشن
- ثاني باراميتر بيكون فيه ال `index` بتاع العنصر الحالي
- ال `forEach` مبتاثرش عالاراي الاصلية لكن ممكن تاثر
- احنا ممكن ناخذ ال `index` بتاع عناصر الاراي ك `parameter` و كمان ممكن تاخذ باراميتر ثالث اللي هو بيكون الاراي الاصلية
- فنقدر نعدل في الاراي الاصلية بالشكل ده

```
let nums = [1,2,1,2];
nums.forEach((element,index,arr) => {
  arr[index] = element * 10;
});
console.log(nums);
```

-
-
-

- ثاني فانكشن معنا هي ال `map`

```
let arr1 = [1,2,3,4,5];
let arr2 = arr1.map(element => element*5);
console.log(arr2);
```

- فايدتها انها بتطبق الفانكشن اللي ادناهاها على الاراي و تخزن الناتج في ارري جديدة
- ال `callback function` بتاخذ باراميتر اللي هو هنا `element` بيكون فيه كل عنصر من الاراي

-
-
-

- ثالث فانكشن هي ال `filter`

```
let arr1 = [1,2,3,4,5];
let arr2 = arr1.filter(element => element >= 3);
```

```
console.log(arr2);
```

- فانكشن `filter` هي بتعمل نسخة جديدة من الارراي لكن متفلترتة على اساس شرط معين
 - طبعا هي بتاخذ `callback function` كبارامتر
 - طب الفانكشن ده بتعمل ايه
 - الفانكشن ال `callback` المفروض ترجع `boolean` يا `true` يا `false` و على اساس ده بتقرر هتاخذ العنصر ده في الارراي الجديدة ولا لا
 - باختصار عشان نعرف نعمل `true` او `false` بنحتاج `condition`
 -
 -
 - رابع فانكشن هي ال `reduce`
 - الاتنين اللي فاتو كانوا بيرجعو ارراي ال `reduce` بترجع قيمة واحدة بس
 - ال `reduce` بتاخذ `callback function` بتاخذ اتنين باراميترز واحد اسمه `accumulator` و واحد اسمه `currentValue`
 - طب كل واحد منهم بيعمل ايه
 - لو مثلا احنا عايزين نجمع الارقام اللي في الارراي هنعمل ايه ؟
 - هنعمل `variable` نسميه `sum` مثلا و نمشي على كل رقم في الارراي نزوده عال `sum` ده
 - و في الآخر هنلاقي الارقام كلها جينا مجموعها في ال `sum` ده
 -
 - نفس الكلام هيجصل هنا بالظبط
 - ال `accumulator` هنا هو ال `sum` اللي هنجمع فيه عناصر الارراي
 - ال `currentValue` هو كل عنصر بنمشي عليه في الارراي
 - طب الفانكشن هتمشي ازاى بالظبط
- هنشتغل عالمثال ده

JAVASCRIPT

```
let nums = [1, 2, 3, 4];
let result = nums.reduce((acc, current) => acc + current, 0);
console.log(result);
```

--> Intialization

acc = 0 No Current Value

--> First Step

acc = 0 Current Value = 1

--> Second Step

acc = 1 Current Value = 2

--> Third Step

acc = 3 Current Value = 3

--> Forth Step

acc = 6

Current Value = 4

--> Fifth Step

acc = 10

No Currrent Value

- زي ما قولنا ال `currentValue` هو كل عنصر من الاراي
- ال `accumulator` ده هو اللي بنجمع فيه
- اول خطوة ال `accumulator` بيتحط فيه القيمة الابتدائية
- بعد كل خطوة بنجمع ال `accumulator` على ال `currentValue` و الناتج بيتحط في ال `accumulator`
- في النهاية اللي بيرجع الناتج الاخير اللي بيكون في ال `accumulator`

٥

Hoisting

- معروف ان js لغة interpreted
- يعني ايه interpreted تاني ؟
- يعني كل سطر بيتنفذ زي ما هو بالترتيب
- طب بناء على ده الكود ده المفروض يطلع ايه ؟

```
console.log(x);  
var x = 0;
```

JAVASCRIPT

- المفروض بالنسبالنا يطلع ايرور
- ليه ؟
- لان ازاى هنطبع ال `x` قبل ما نتعرف
- احنا اصلا معدناش `x` فهنطبع ايه
- ده منطق صح مية في المية بس المفاجأة ان الكود ده مش هيطلع ايرور ولا حاجة
- ولا حتى هيطلع قيمة `x` هو هيطلع `undefined`
- طب ليه اصلا هو يعرف منين ان في `x` موجودة بعد جملة الطباعة بشوية
- ما لو معرفناش ال `x` اصلا قبلها ولا بعدها هيطلع ايرور

- السبب ان js فيها حاجة اسمها hoisting و هي بتعمل declaration لكل ال variables في اول الفايل
- و المكان اللي احنا بنعرفها فيه بقى الحقيقي هي بتحط قيمة لل variable ده

- يعني الكود بالنسبة ل js سيكون كدة

JAVASCRIPT

```
var x;
console.log(x);
x = 0;
```

- بتعرف ال x في الاول بس مبتديلوش قيمة فبيبقى جواه undefined لوحده
- فلما نيجي نطبعه هيطلعنا undefined

•

- الكلام ده شغال في var بس const , let مبيحصلهمش hoisting
- المكان اللي بيتعرفو فيه بيشتغلو فيه لو استخدمناهم قبل التعريف بيطلع ايرور

•

•

- طب هو ده بيحصل في ال variables بس ؟

- لا بيحصل في الفانكشن بردو

- بمعنى احنا ما دام عرفنا الفانكشن يبقى نقدر نستدعيها في اي حنة براحتنا

JAVASCRIPT

```
printHello(); // Hello
function printHello() {
    console.log("Hello");
}
printHello(); // Hello
```

- استدعاء الفانكشن في المرتين هيطلع الناتج صح و مش هيطلع ايرور ليه لان js ترجمت الكود بطريقة تانية
- هي خدت تعريف الفانكشن حطته في الاول خالص و بعدين بقى كل الاستدعاءات بقت تحته
- يعني js قرت الكود كدة

JAVASCRIPT

```
function printHello() {
    console.log("Hello");
}
printHello(); // Hello
printHello(); // Hello
```

- طب ناتج الكود اللي جاي ده ايه ؟

JAVASCRIPT

```
printHello();
var printHello = function () {
    console.log("Hello");
}
```

```
printHello();
```

- الاجابة ان الاستدعاء الاول هيطلع ايرور
- طب ليه مش هي محطوطة variable من نوع `var` و المفروض ال variable ده يتعمله hoisting و يتعرف بدري
- ادبك قولت يتعرف بدري
- الايرور اللي هيطلعنا هو `printHello is not a function`
- اصلا ؟ طب ازاي ما ال variable بعد ما حطينا فيها تعريف الفانكشن بقي فانكشن
- بس اللي بيحصل لل variable انه بيتعرف بس فوق مش بيبقى فيه اي قيمة بيبقى جواه `undefined`
- و بعدها بقي احنا بنحط جواه الفانكشن
- بس قبل ما نحط جواه الفانكشن مينفعش ابدأ نتعامل معاه كفانكشن
- بمعنى مينفعش نكتب اسمه و جمبه القوسين بتوع استدعاء الفانكشن لان هو دلوقتي لسة variable عادي مش فانكشن
-
- بالنسبة للاستدعاء الثاني هيطلع الناتج صح عادي و هيستدعي الفانكشن
-
- في حد ممكن يسأل طب ليه الفانكشن هي كمان متعرفتش فوق زي ما كنا بنقول في الاول
- ببساطة الفانكشن كانها مش موجودة اصلا لحد ما نعمل initialize لل variable
- ليه لان هي متعرفتش غير وقت ال initialize فبالناتالي مش هينفع تتعرف فوق لان تعريفها نفسه مربوط بالمكان اللي ال variable خد قيمة فيه



Object

- لو عايزين نخزن مثلا بيانات عن شخص (اسم-سن-نوع-وظيفة) هنخزنها ازاي
- التفكير العادي هيقول اتنا نخزن كل حاجة في variable

JAVASCRIPT

```
let name = "Ahmed";
let age = 23;
let gender = "Male";
let job = "Backend Developer";

let name1 = "Sara";
let age1 = 20;
let gender1 = "Female";
let job1 = "Frontend Developer";
```

- طب مين يقول ان ال variables ده بتاعت شخص واحد
- كل variable مستقل و ملوش علاقة بالباقي
- طب الحل الثاني اننا نجمعهم في ارراي

```
JAVASCRIPT
let person = ["Ahmed", 23, "Male", "Backend Developer"];
let person1 = ["Sara", 20, "Female", "Frontend Developer"];
```

- طب هو ده طبيعي يعني المفروض نعرف مين ان المكان الاول في الارراي ده بيكون فيه الاسم
- و المكان الثاني بيكون فيه السن و هكذا
- من هنا طلع ال object
- ال object جمعلنا البيانات و خلاهم متعلقين ببعض و جمعهم في حنة واحدة
- و كل معلومة بنعوزها في ال اوبجيكت ده بنجيبها باسمها

```
JAVASCRIPT
let person = {
  name: "Ahmed",
  age: 23,
  gender: "Male",
  job: "Backend Developer"
}

let person1 = {
  name: "Sara",
  age: 20,
  gender: "Female",
  job: "Frontend Developer"
}
```

- ال name , age , gender , job دول اسمهم properties يعني الخصائص بتاعت ال object
- عايزين نطبع ال اوبجيكت كله هنطبعه عادي زي اي variable

```
JAVASCRIPT
console.log(person);
```

- لو عايزين نجيب قيمة property معينة عندنا طريقتين

```
JAVASCRIPT
console.log(person.name);
console.log(person["name"]);
```

- طب ازاى اضيف property لل object

```
JAVASCRIPT
person.city = "Cairo";
```

- طب لو عايز اعدل في قيمة property من اللي موجودين
- نفس الطريقة برودو

```
person.age = 24;
```

JAVASCRIPT

- طب لو حابين نمسح property او method
- ال delete بتمسح ال property بال value بتاعتها

```
delete person.gender;
```

JAVASCRIPT

- عندنا حاجة تانية بتكون موجودة في ال object غير ال properties بتاعته
- هي ال actions اللي ممكن نعملها عال object ده اسمها methods

```
let person = {
  name: "Ahmed",
  age: 23,
  gender: "Male",
  job: "Backend Developer",
  introduce: function () {
    let intro = `Hello,
    I am ${this.name} and I am a ${this.job}`;
    console.log(intro);
  },
}
```

JAVASCRIPT

- بس بنستدعي ال method بشكل مختلف لازم نحط جنبها الاقواس بتاعت استدعاء الفانكشن

```
person.introduce();
```

JAVASCRIPT

- طب ايه this اللي كتبناها جوا ال method ده ؟
- ده بتشير للاوبجيكت نفسه يعني هي اتكتبت جوا object اسمه person يبقى هي بتشير للاوبجيكت اللي اسمه person
- يعني بما اننا كتبنا this.name يبقى احنا نقصد الاسم اللي جوا ال person
- ال this طول ما هي جوا ال object فهي بتشاور عليه يعني مينفعش نستخدمها برا لان هنلاقيها بتشاور على حاجات تانية
- غير ال object

•

- دلوقتي احنا حلينا مشكلة اننا نجمع بيانات شخص واحد مع بعضها
- بس برودو لو عملنا object لكل طالب هم لسة منفصلين عن بعض ازاى نجتمعهم مع بعض
- الحل اننا نحط ال objects ده في array مع بعضهم بس

```
let people = [
  {
    name: "Ahmed",
    age: 23,
    gender: "Male",
    job: "Backend Developer",
    introduce: function () {
      let intro = `Hello,
      I am ${this.name} and I am a ${this.job}`;
      console.log(intro);
    },
  },
  {
    name: "Sara",
    age: 20,
    gender: "Female",
    job: "Frontend Developer",
    introduce: function () {
      let intro = `Hello,
      I am ${this.name} and I am a ${this.job}`;
      console.log(intro);
    },
  }
];
```

- ممكن برده نعمل nested object
- بمعنى يبقى عندنا object جواه object ثاني جواه object ثالث و هكذا
- و ممكن ال property تكون عبارة عن ارراي

```
let students = [
  {
    name: {
      firstName: "Ahmad",
      lastName: "Mohammed"
    },
    age: 15,
    gender: "Male",
    courses: ["Math", "Science"],
    introduce: function () {
      let intro = `Hello,
      I am ${this.name} and I am ${this.age} years old`;
      console.log(intro);
    },
  },
];
```

```
}  
];
```



Destructuring

- الاسهل بدل ما نقعد نكتب كل شوية `person.name` او `person.age` اننا نحطهم في variable

JAVASCRIPT

```
let person = {  
  name: "Ahmed",  
  age: 23,  
  gender: "Male",  
  job: "Backend Developer",  
  introduce: function () {  
    let intro = `Hello,  
    I am ${this.name} and I am a ${this.job}`;  
    console.log(intro);  
  },  
}  
  
let name = person.name;  
let age = person.age;  
  
console.log(name, age);
```

- بردو لو عدد ال variables كبير هتبقى ده طريقة صعبة اوي و هتاخذ وقت في الكتابة
- عشان كدة عندنا ال destructuring سهل الدنيا اكر بقت كدة

JAVASCRIPT

```
let {name, age} = person;  
console.log(name, age);
```

- هو استخرج القيم اللي هنتحط في ال variables من ال object
- ممكن ناخذ ال properties كلها بالطريقة ده و ممكن لا عادي
- ترتيب ال variables مش هيفرق
- لازم اسماء ال variables تكون نفس اسماء ال attributes
- لو عايزين اسماء مختلفة يبقى نعمل كدة

```
let {name: myName, age: myAge} = person;
console.log(myName, myAge);
```

- ال `name` , `age` هي properties
- ال `myName` , `myAge` هي variables
- كدة احنا عملنا destructuring لل object

-

- ممكن نعمل destructuring لل array

```
let nums = [1,2,3,4,5];

[num1, num2] = nums;
console.log(num1, num2);
```

```
let nums = [1,2,3,4,5];

[num1, num2, ...rest] = nums;
console.log(num1, num2, rest);
```

- ال `num1` هتاخذ اول عنصر
- ال `num2` هتاخذ ثاني عنصر
- ال `rest` هيتجمع فيها باقي العناصر في ارایي جديدة
- ال `...` اسمها rest operator هي بتجمع باقي القيم في ارایي
- استخدمناها قبل كدة في الفانكشن فاكرين ؟